

Erick Molina Gehring

## **Manipulação de Processos**

Relatório técnico de atividade prática solicitado pelo professor Rodrigo Campiolo na disciplina de Sistemas Operacionais do Bacharelado em Ciência da Computação da Universidade Tecnológica Federal do Paraná.

Universidade Tecnológica Federal do Paraná – UTFPR

Departamento Acadêmico de Computação – DACOM

Bacharelado em Ciência da Computação – BCC

Campo Mourão

Abril / 2026

# Sumário

|   |                                        |   |
|---|----------------------------------------|---|
| 1 | Introdução . . . . .                   | 3 |
| 2 | Objetivos . . . . .                    | 3 |
| 3 | Materiais . . . . .                    | 3 |
| 4 | Procedimentos e Resultados . . . . .   | 3 |
|   | 4.1 Manipulação de Processos . . . . . | 3 |
|   | 4.2 Programação . . . . .              | 7 |
| 5 | Discussão dos Resultados . . . . .     | 7 |
|   | 5.1 Parte 1 . . . . .                  | 7 |
|   | 5.2 Parte 2 . . . . .                  | 8 |
| 6 | Conclusões . . . . .                   | 8 |

# 1 Introdução

Este relatório apresenta as atividades desenvolvidas no Laboratório 2 da disciplina de Sistemas Operacionais, cujo tema é a manipulação de processos em ambiente **Linux**. O laboratório foi dividido em duas partes: a primeira voltada à observação e compreensão do comportamento de processos no sistema operacional, e a segunda à implementação de programas em **C++** que exploram os mecanismos de criação e gerenciamento de processos.

O objetivo principal foi compreender a estrutura de processos em Linux, identificando conceitos como **daemons**, **processos interativos**, **processos zumbis** e **hierarquias de processos**. Para isso, foram utilizadas ferramentas como **ps**, **top** e **pstree**, além das chamadas de sistema **fork()**, **execvp()** e **wait()** para implementar os programas da Parte 2.

# 2 Objetivos

O objetivo desta atividade é aprender a compreender a estrutura de processos em Linux e manipular processos em linguagem de programação.

# 3 Materiais

- Distribuição Linux.
- Ambiente de desenvolvimento.
- Comandos do sistema e bibliotecas de programação.

# 4 Procedimentos e Resultados

## 4.1 Manipulação de Processos

1. Execute o comando **ps aux** e identifique três processos do sistema (daemons) e três processos interativos. Explique os valores da saída desse comando para um dos processos identificados.

Resposta:

Tabela 1 – Três processos **daemons** identificados

| USER     | PID | COMMAND              | Porque é daemon?                          |
|----------|-----|----------------------|-------------------------------------------|
| avahi    | 948 | avahi-daemon         | Serviço mDNS/DNS-SD de rede local         |
| root     | 953 | /usr/sbin/cron       | Agendador de tarefas do sistema           |
| message+ | 955 | /usr/bin/dbus-daemon | Barramento de comunicação entre processos |

Tabela 2 – Três processos **interativos** identificados

| USER  | PID   | TTY   | COMMAND       |
|-------|-------|-------|---------------|
| erick | 5612  | pts/0 | /usr/bin/bash |
| erick | 6291  | pts/1 | /usr/bin/bash |
| erick | 16968 | pts/2 | bash          |

Tabela 3 – Explicação detalhada do processo **cron** (PID 953)

| Campo   | Valor          | Significado                                                                            |
|---------|----------------|----------------------------------------------------------------------------------------|
| USER    | root           | Processo pertence ao superusuário                                                      |
| PID     | 953            | Identificador único do processo                                                        |
| %CPU    | 0.0            | Não está consumindo CPU no momento                                                     |
| %MEM    | 0.0            | Consumo de memória RAM desprezível                                                     |
| VSZ     | 6792 KB        | Tamanho total da memória virtual alocada                                               |
| RSS     | 2848 KB        | Memória física efetivamente em uso                                                     |
| TTY     | ?              | Sem terminal associado                                                                 |
| STAT    | Ss             | S (INTERRUPTIBLE SLEEP): esperando por um evento; s = líder de sessão (session leader) |
| START   | 14:55          | Foi iniciado às 14h55                                                                  |
| TIME    | 0:00           | Consumiu menos de 1 segundo de CPU no total                                            |
| COMMAND | /usr/sbin/cron | Caminho completo do executável                                                         |

2. Há processos **zombies** executando em seu sistema operacional? Posso eliminá-los do sistema usando o comando **kill -SIGKILL pid\_zombie**? Justifique.

Resposta:

Tabela 4 – Processo Zumbi

| USER  | PID   | %CPU | %MEM | VSZ | RSS | TTY | STAT | START | TIME | COMMAND        |
|-------|-------|------|------|-----|-----|-----|------|-------|------|----------------|
| erick | 12376 | 0.0  | 0.0  | 0   | 0   | ?   | Z    | 15:04 | 0:00 | [sd_espeak-ng] |

Sim, O **estado Z confirma o zumbi**. Ele é filho do speech-dispatcher (PID 12364). Não é possível eliminá-lo com **kill -SIGKILL 12376** porque o processo já terminou sua execução, ele não existe mais como processo ativo, apenas como uma entrada na tabela de processos aguardando que o pai chame wait() para recolher seu código de saída. Como não está em execução, nenhum sinal tem efeito sobre ele. A solução seria **encerrar o processo pai (PID 12364)**, fazendo o zumbi ser adotado pelo **systemd**, que faz o recolhimento automaticamente.

3. Quais os processos com maior utilização de CPU? Quais os processos com maior utilização de memória? Qual o processo do usuário está a mais tempo em execução?

Resposta:

Tabela 5 – Maior uso de **CPU**

| PID  | %CPU  | COMMAND     |
|------|-------|-------------|
| 3471 | 31.8% | Firefox     |
| 4206 | 22.1% | VS Code     |
| 2589 | 16.0% | gnome-shell |

Tabela 6 – Maior uso de **memória**

| PID   | %MEM | RSS    | COMMAND |
|-------|------|--------|---------|
| 3471  | 4.5% | 742 MB | Firefox |
| 12412 | 3.6% | 602 MB | Firefox |
| 3825  | 3.4% | 566 MB | Firefox |

Tabela 7 – Processo do usuário a mais **tempo** em execução

| PID  | TIME | COMMAND     |
|------|------|-------------|
| 3471 | 4:52 | Firefox     |
| 4206 | 3:22 | VS Code     |
| 2589 | 2:28 | gnome-shell |

4. Como eu faço para suspender um processo no Linux? Como eu faço para retomar a execução novamente?

Resposta:

- Suspende:

– **Ctrl + Z** ou **kill -STOP PID**

- Retomar:
  - Em foreground:
    - \* **fg %1**
  - Em background:
    - \* **bg %1**

5. O que aconteceria se um processo criasse recursivamente processos filhos indefinidamente? Implemente um programa em Linux que faça isso e apresente o resultado. (Sugestão: testar na máquina virtual).

Resposta:

Se um processo criasse filhos recursivamente e indefinidamente (**fork bomb**), o sistema sofreria uma negação de serviço local. Cada chamada a **fork()** duplica o processo, gerando uma sobrecarga exponencial:  $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \dots$ . Em poucos segundos os recursos do sistema seriam esgotados:

- A tabela de processos do kernel ficaria cheia (**limite de PIDs**);
- A **memória RAM** seria consumida pelas cópias dos processos;
- A **CPU** ficaria 100% ocupada apenas gerenciando os processos;
- O sistema ficaria completamente sem resposta.

Um código simples que serviria para esse exemplo seria algo como:

Listing 1 – Fork Bomb Recursivo

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void fork_bomb() {
    fork();           // cria um filho
    fork_bomb();      // pai E filho chamam recursivamente
}

int main() {
    fork_bomb();
    return 0;
}
```

Porém, como não estou com uma máquina virtual configurada no meu computador, que seria o ideal para testar o código 1 acima, fiz outro código com limitações para

efetuar os testes e colocando um limite de profundidade da recursão de 4 como é demonstrado na figura 1, o resultado foi o seguinte:

```
erick@sigmacomputer:~/Documentos/BCC/5ºSemestre/SO - Sistemas Operacionais/Laboratório 2$ gcc -o forkbomb parte1-ex5.c
erick@sigmacomputer:~/Documentos/BCC/5ºSemestre/SO - Sistemas Operacionais/Laboratório 2$ ./forkbomb
=== Fork Bomb Recursiva (limitada a 4 níveis) ===
PID inicial: 523546
Total esperado de processos filhos: 15

Nível 1 | PID=523547 criado pelo pai PID=523546
Nível 1 | PID=523548 criado pelo pai PID=523546
Nível 2 | PID=523549 criado pelo pai PID=523547
Nível 2 | PID=523550 criado pelo pai PID=523548
Nível 3 | PID=523552 criado pelo pai PID=523549
Nível 2 | PID=523553 criado pelo pai PID=523548
Nível 3 | PID=523554 criado pelo pai PID=523550
Nível 3 | PID=523555 criado pelo pai PID=523549
Nível 4 | PID=523556 criado pelo pai PID=523552
Nível 3 | PID=523557 criado pelo pai PID=523553
Nível 3 | PID=523558 criado pelo pai PID=523550
Nível 2 | PID=523551 criado pelo pai PID=523547
Nível 4 | PID=523559 criado pelo pai PID=523552
Nível 3 | PID=523561 criado pelo pai PID=523553
Nível 4 | PID=523560 criado pelo pai PID=523554
Nível 4 | PID=523563 criado pelo pai PID=523557
Nível 4 | PID=523564 criado pelo pai PID=523555
Nível 4 | PID=523565 criado pelo pai PID=523558
Nível 4 | PID=523568 criado pelo pai PID=523561
Nível 4 | PID=523566 criado pelo pai PID=523554
Nível 3 | PID=523567 criado pelo pai PID=523551
Nível 4 | PID=523569 criado pelo pai PID=523557
Nível 4 | PID=523570 criado pelo pai PID=523558
Nível 4 | PID=523572 criado pelo pai PID=523561
Nível 4 | PID=523562 criado pelo pai PID=523555
Nível 4 | PID=523573 criado pelo pai PID=523567
Nível 4 | PID=523574 criado pelo pai PID=523567
Nível 3 | PID=523571 criado pelo pai PID=523551
Nível 4 | PID=523575 criado pelo pai PID=523571
Nível 4 | PID=523576 criado pelo pai PID=523571
```

Figura 1 – Fork Bomb Recursivo Limitado

## 4.2 Programação

Os programas feitos para cada um dos exercícios descritos na parte 2 da atividade foram feitos e executados com sucesso e serão enviados juntos deste relatório.

Será enviado também o código feito para a questão 5 da parte 1, no qual quando executado gerou o a figura 1.

## 5 Discussão dos Resultados

### 5.1 Parte 1

A análise do comando **ps aux** permitiu identificar claramente a diferença entre processos de sistema (daemons) e processos interativos. Daemons como **cron**, **avahi-daemon** e **dbus-daemon** apresentaram o campo **TTY** igual a **?**, indicando que não estão associados a nenhum terminal, enquanto os processos interativos como o **bash** estavam vinculados a terminais **pts/0**, **pts/1** e **pts/2**. Foi também identificado um processo zumbi real no sistema — o processo **sd\_espeak-ng** (PID 12376) com estado **Z** — confirmando

na prática que processos nesse estado não podem ser eliminados com **kill -SIGKILL**, pois já encerraram sua execução e aguardam apenas que o pai chame **wait()**.

A análise de uso de **CPU** e **memória** revelou que o **Firefox** e o **VS Code** foram os processos do usuário com maior consumo de recursos, com o Firefox acumulando o maior tempo de CPU. A demonstração da **fork bomb**, realizada com limite de profundidade para não comprometer o sistema, evidenciou o crescimento exponencial de processos gerado pela criação recursiva, ilustrando como um sistema sem controle de limites seria rapidamente esgotado em termos de memória, CPU e entradas na tabela de processos do kernel.

## 5.2 Parte 2

Todos os programas implementados foram compilados e executados com sucesso. O programa de hierarquia de processos (**hierarquia.cpp**) criou corretamente a árvore binária de processos com N níveis, exibindo a estrutura via **pstree**. O programa **executa\_comando.cpp** demonstrou o uso de **execvp()** para substituir a imagem do processo filho pelo comando informado, com o pai aguardando corretamente o retorno via **waitpid()**. O programa de busca paralela (**busca\_vetor.cpp**) distribuiu o vetor entre N filhos e realizou a busca de forma concorrente, com cada filho reportando seu **PID** e as posições encontradas. Por fim, o shell simples (**shell.cpp**) implementou corretamente o suporte a comandos em foreground e background, recolhendo filhos encerrados automaticamente para evitar processos zumbis.

## 6 Conclusões

A atividade permitiu consolidar na prática os conceitos teóricos sobre gerenciamento de processos em Linux. A observação direta do sistema com ferramentas como **ps**, **aux** e a identificação de um processo zumbi real tornaram o aprendizado mais concreto e significativo. A implementação dos programas em C++ reforçou o entendimento das chamadas de sistema fundamentais, **fork()**, **execvp()** e **wait()**, e de como o sistema operacional gerencia a criação, execução e término de processos.

A atividade da **fork bomb** foi especialmente relevante para compreender os riscos de processos sem controle de recursos e a importância dos limites impostos pelo kernel.